

Laboratorio 6

Ignacio Méndez Alvarez (22613) y Diego Soto Flores (22737)

Enlace al repositorio: <https://github.com/ignaciomendeza/VPC-LAB6.git>

```
In [1]: import os
import time
import copy
import random
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim

from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, Subset
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, classification_report
```

Carga datos

```
In [2]: import kagglehub

# Download latest version
path = kagglehub.dataset_download("aryashah2k/mango-leaf-disease-dataset")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'mango-leaf-disease-dataset' dataset.
Path to dataset files: /kaggle/input/mango-leaf-disease-dataset

```
In [3]: from torchvision import transforms

transform_train = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.RandomResizedCrop(224),
    transforms.ToTensor()
])

transform_test = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

```
In [4]: import os
```

```
from torchvision import datasets

print("Ruta:", path)
print("Contenido:", os.listdir(path))

dataset = datasets.ImageFolder(path, transform=transform_train)

print("Total imágenes:", len(dataset))
print("Clases:", dataset.classes)
print("Número de clases:", len(dataset.classes))
```

Ruta: /kaggle/input/mango-leaf-disease-dataset

Contenido: ['Powdery Mildew', 'Cutting Weevil', 'Anthracnose', 'Bacterial Canker', 'Sooty Mould', 'Gall Midge', 'Healthy', 'Die Back']

Total imágenes: 4000

Clases: ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Gall Midge', 'Healthy', 'Powdery Mildew', 'Sooty Mould']

Número de clases: 8

```
In [5]: def set_seed(seed=42):
        random.seed(seed)
        np.random.seed(seed)
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

        set_seed(42)

        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        print("Device:", device)
```

Device: cuda

```
In [6]: imagenet_mean = [0.485, 0.456, 0.406]
        imagenet_std = [0.229, 0.224, 0.225]

        transform_train = transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.RandomHorizontalFlip(),
            transforms.RandomRotation(20),
            transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
            transforms.ToTensor(),
            transforms.Normalize(mean=imagenet_mean, std=imagenet_std)
        ])

        transform_eval = transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(mean=imagenet_mean, std=imagenet_std)
        ])
```

```
In [18]: imagenet_mean = [0.485, 0.456, 0.406]
         imagenet_std = [0.229, 0.224, 0.225]

         transform_train_inception = transforms.Compose([
             transforms.Resize((299, 299)),
             transforms.RandomHorizontalFlip(),
             transforms.RandomRotation(20),
```

```

        transforms.RandomResizedCrop(299, scale=(0.8, 1.0)),
        transforms.ToTensor(),
        transforms.Normalize(mean=imagenet_mean, std=imagenet_std)
    ])

transform_eval_inception = transforms.Compose([
    transforms.Resize((299, 299)),
    transforms.ToTensor(),
    transforms.Normalize(mean=imagenet_mean, std=imagenet_std)
])

```

Split

```

In [7]: full_dataset_for_labels = datasets.ImageFolder(path)
        class_names = full_dataset_for_labels.classes
        num_classes = len(class_names)

        print("Clases:", class_names)
        print("Número de clases:", num_classes)
        print("Total imágenes:", len(full_dataset_for_labels))

```

Clases: ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Gall Midge', 'Healthy', 'Powdery Mildew', 'Sooty Mould']
 Número de clases: 8
 Total imágenes: 4000

```

In [8]: targets = full_dataset_for_labels.targets
        indices = np.arange(len(targets))

        train_idx, temp_idx = train_test_split(
            indices,
            test_size=0.30,
            stratify=targets,
            random_state=42
        )

        temp_targets = [targets[i] for i in temp_idx]

        val_idx, test_idx = train_test_split(
            temp_idx,
            test_size=0.50,
            stratify=temp_targets,
            random_state=42
        )

        print("Train:", len(train_idx))
        print("Validation:", len(val_idx))
        print("Test:", len(test_idx))

```

Train: 2800
 Validation: 600
 Test: 600

Datasets y dataloaders

```
In [9]: train_dataset_full = datasets.ImageFolder(path, transform=transform_train)
eval_dataset_full = datasets.ImageFolder(path, transform=transform_eval)

train_data = Subset(train_dataset_full, train_idx)
val_data = Subset(eval_dataset_full, val_idx)
test_data = Subset(eval_dataset_full, test_idx)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True, num_workers=2, p
val_loader = DataLoader(val_data, batch_size=32, shuffle=False, num_workers=2, pin_
test_loader = DataLoader(test_data, batch_size=32, shuffle=False, num_workers=2, pi

print("Train batches:", len(train_loader))
print("Val batches:", len(val_loader))
print("Test batches:", len(test_loader))
```

Train batches: 88

Val batches: 19

Test batches: 19

```
In [19]: train_dataset_inception_full = datasets.ImageFolder(path, transform=transform_train)
eval_dataset_inception_full = datasets.ImageFolder(path, transform=transform_eval_i

train_data_inception = Subset(train_dataset_inception_full, train_idx)
val_data_inception = Subset(eval_dataset_inception_full, val_idx)
test_data_inception = Subset(eval_dataset_inception_full, test_idx)

train_loader_inception = DataLoader(train_data_inception, batch_size=32, shuffle=Tr
val_loader_inception = DataLoader(val_data_inception, batch_size=32, shuffle=False,
test_loader_inception = DataLoader(test_data_inception, batch_size=32, shuffle=Fals

print("Train Inception:", len(train_data_inception))
print("Val Inception:", len(val_data_inception))
print("Test Inception:", len(test_data_inception))
```

Train Inception: 2800

Val Inception: 600

Test Inception: 600

```
In [10]: def freeze_backbone(model):
        for param in model.parameters():
            param.requires_grad = False
```

Carga ResNet50

```
In [11]: from torchvision.models import ResNet50_Weights, Inception_V3_Weights, MobileNet_V2

resnet = models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
freeze_backbone(resnet)
resnet.fc = nn.Linear(resnet.fc.in_features, num_classes)
resnet = resnet.to(device)
```

```
print("ResNet cargado")
```

Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth

100%|██████████| 97.8M/97.8M [00:00<00:00, 194MB/s]

ResNet cargado

Carga InceptionV3

```
In [12]: inception = models.inception_v3(weights=Inception_V3_Weights.IMAGENET1K_V1, aux_log
freeze_backbone(inception)
```

```
inception.fc = nn.Linear(inception.fc.in_features, num_classes)
inception.AuxLogits.fc = nn.Linear(inception.AuxLogits.fc.in_features, num_classes)
```

```
inception = inception.to(device)
```

```
print("Inception cargado")
```

Downloading: "https://download.pytorch.org/models/inception_v3_google-0cc3c7bd.pth" to /root/.cache/torch/hub/checkpoints/inception_v3_google-0cc3c7bd.pth

100%|██████████| 104M/104M [00:00<00:00, 175MB/s]

Inception cargado

Carga MobileNetV2

```
In [13]: mobilenet = models.mobilenet_v2(weights=MobileNet_V2_Weights.IMAGENET1K_V2)
freeze_backbone(mobilenet)
mobilenet.classifier[1] = nn.Linear(mobilenet.classifier[1].in_features, num_classes)
mobilenet = mobilenet.to(device)
```

```
print("MobileNet cargado")
```

Downloading: "https://download.pytorch.org/models/mobilenet_v2-7ebf99e0.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v2-7ebf99e0.pth

100%|██████████| 13.6M/13.6M [00:00<00:00, 76.1MB/s]

MobileNet cargado

```
In [14]: def count_trainable_params(model):
return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

```
print("Trainable ResNet:", count_trainable_params(resnet))
print("Trainable Inception:", count_trainable_params(inception))
print("Trainable MobileNet:", count_trainable_params(mobilenet))
```

Trainable ResNet: 16392

Trainable Inception: 22544

Trainable MobileNet: 10248

Entrenamiento

```
In [15]: def train_model(model, train_loader, val_loader, model_name, epochs=15, patience=3,
criterion = nn.CrossEntropyLoss())
```

```
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), 1

best_val_loss = float("inf")
best_state = copy.deepcopy(model.state_dict())
history = {
    "train_loss": [],
    "val_loss": [],
    "val_acc": [],
    "val_f1": []
}

patience_counter = 0

for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        if model_name == "Inception":
            outputs, aux_outputs = model(images)
            loss_main = criterion(outputs, labels)
            loss_aux = criterion(aux_outputs, labels)
            loss = loss_main + 0.4 * loss_aux
        else:
            outputs = model(images)
            loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    train_loss = running_loss / len(train_loader.dataset)

    model.eval()
    val_running_loss = 0.0
    y_true = []
    y_pred = []

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            if isinstance(outputs, tuple):
                outputs = outputs[0]

            loss = criterion(outputs, labels)
            val_running_loss += loss.item() * images.size(0)
```

```
        preds = torch.argmax(outputs, dim=1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

    val_loss = val_running_loss / len(val_loader.dataset)
    val_acc = accuracy_score(y_true, y_pred)
    val_f1 = f1_score(y_true, y_pred, average="macro")

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)
    history["val_f1"].append(val_f1)

    print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_state = copy.deepcopy(model.state_dict())
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f"Early stopping activado en epoch {epoch+1}")
            break

    model.load_state_dict(best_state)
    return model, history
```

Entrenar ResNet

```
In [16]: resnet, history_resnet = train_model(
    resnet,
    train_loader,
    val_loader,
    model_name="ResNet",
    epochs=15,
    patience=3,
    lr=1e-3
)
```

```
Epoch 1/15 | Train Loss: 1.0547 | Val Loss: 0.5363 | Val Acc: 0.9333 | Val F1: 0.9293
Epoch 2/15 | Train Loss: 0.3895 | Val Loss: 0.2827 | Val Acc: 0.9683 | Val F1: 0.9675
Epoch 3/15 | Train Loss: 0.2376 | Val Loss: 0.1977 | Val Acc: 0.9767 | Val F1: 0.9762
Epoch 4/15 | Train Loss: 0.1680 | Val Loss: 0.1585 | Val Acc: 0.9783 | Val F1: 0.9779
Epoch 5/15 | Train Loss: 0.1325 | Val Loss: 0.1152 | Val Acc: 0.9850 | Val F1: 0.9849
Epoch 6/15 | Train Loss: 0.1096 | Val Loss: 0.1028 | Val Acc: 0.9850 | Val F1: 0.9849
Epoch 7/15 | Train Loss: 0.0905 | Val Loss: 0.0817 | Val Acc: 0.9883 | Val F1: 0.9883
Epoch 8/15 | Train Loss: 0.0822 | Val Loss: 0.0723 | Val Acc: 0.9917 | Val F1: 0.9917
Epoch 9/15 | Train Loss: 0.0748 | Val Loss: 0.0651 | Val Acc: 0.9950 | Val F1: 0.9950
Epoch 10/15 | Train Loss: 0.0636 | Val Loss: 0.0581 | Val Acc: 0.9933 | Val F1: 0.9933
Epoch 11/15 | Train Loss: 0.0599 | Val Loss: 0.0587 | Val Acc: 0.9917 | Val F1: 0.9916
Epoch 12/15 | Train Loss: 0.0551 | Val Loss: 0.0472 | Val Acc: 0.9950 | Val F1: 0.9950
Epoch 13/15 | Train Loss: 0.0473 | Val Loss: 0.0409 | Val Acc: 0.9950 | Val F1: 0.9950
Epoch 14/15 | Train Loss: 0.0473 | Val Loss: 0.0454 | Val Acc: 0.9917 | Val F1: 0.9917
Epoch 15/15 | Train Loss: 0.0433 | Val Loss: 0.0385 | Val Acc: 0.9933 | Val F1: 0.9933
```

Entrenar Inception

```
In [20]: inception, history_inception = train_model(
        inception,
        train_loader_inception,
        val_loader_inception,
        model_name="Inception",
        epochs=15,
        patience=3,
        lr=1e-3
    )
```



```
Epoch 1/15 | Train Loss: 1.4241 | Val Loss: 0.6302 | Val Acc: 0.9367 | Val F1: 0.9370
Epoch 2/15 | Train Loss: 0.5344 | Val Loss: 0.3694 | Val Acc: 0.9567 | Val F1: 0.9555
Epoch 3/15 | Train Loss: 0.3861 | Val Loss: 0.2667 | Val Acc: 0.9733 | Val F1: 0.9731
Epoch 4/15 | Train Loss: 0.2852 | Val Loss: 0.2196 | Val Acc: 0.9650 | Val F1: 0.9649
Epoch 5/15 | Train Loss: 0.2461 | Val Loss: 0.1743 | Val Acc: 0.9750 | Val F1: 0.9748
Epoch 6/15 | Train Loss: 0.2166 | Val Loss: 0.1543 | Val Acc: 0.9750 | Val F1: 0.9748
Epoch 7/15 | Train Loss: 0.1959 | Val Loss: 0.1383 | Val Acc: 0.9833 | Val F1: 0.9833
Epoch 8/15 | Train Loss: 0.1816 | Val Loss: 0.1331 | Val Acc: 0.9733 | Val F1: 0.9731
Epoch 9/15 | Train Loss: 0.1817 | Val Loss: 0.1190 | Val Acc: 0.9750 | Val F1: 0.9748
Epoch 10/15 | Train Loss: 0.1740 | Val Loss: 0.1019 | Val Acc: 0.9817 | Val F1: 0.9815
Epoch 11/15 | Train Loss: 0.1524 | Val Loss: 0.0944 | Val Acc: 0.9817 | Val F1: 0.9816
Epoch 12/15 | Train Loss: 0.1534 | Val Loss: 0.1053 | Val Acc: 0.9817 | Val F1: 0.9816
Epoch 13/15 | Train Loss: 0.1397 | Val Loss: 0.0880 | Val Acc: 0.9800 | Val F1: 0.9799
Epoch 14/15 | Train Loss: 0.1470 | Val Loss: 0.0839 | Val Acc: 0.9833 | Val F1: 0.9832
Epoch 15/15 | Train Loss: 0.1319 | Val Loss: 0.0732 | Val Acc: 0.9867 | Val F1: 0.9866
```

Entrenar MobileNet

```
In [22]: mobilenet, history_mobilenet = train_model(
        mobilenet,
        train_loader,
        val_loader,
        model_name="MobileNet",
        epochs=15,
        patience=3,
        lr=1e-3
    )
```

```
Epoch 1/15 | Train Loss: 0.9640 | Val Loss: 0.4582 | Val Acc: 0.9483 | Val F1: 0.9465
Epoch 2/15 | Train Loss: 0.3300 | Val Loss: 0.2581 | Val Acc: 0.9667 | Val F1: 0.9665
Epoch 3/15 | Train Loss: 0.2156 | Val Loss: 0.1811 | Val Acc: 0.9767 | Val F1: 0.9765
Epoch 4/15 | Train Loss: 0.1657 | Val Loss: 0.1450 | Val Acc: 0.9850 | Val F1: 0.9848
Epoch 5/15 | Train Loss: 0.1341 | Val Loss: 0.1099 | Val Acc: 0.9867 | Val F1: 0.9867
Epoch 6/15 | Train Loss: 0.1186 | Val Loss: 0.0978 | Val Acc: 0.9883 | Val F1: 0.9883
Epoch 7/15 | Train Loss: 0.1000 | Val Loss: 0.0945 | Val Acc: 0.9817 | Val F1: 0.9816
Epoch 8/15 | Train Loss: 0.0905 | Val Loss: 0.0759 | Val Acc: 0.9900 | Val F1: 0.9900
Epoch 9/15 | Train Loss: 0.0767 | Val Loss: 0.0720 | Val Acc: 0.9833 | Val F1: 0.9833
Epoch 10/15 | Train Loss: 0.0747 | Val Loss: 0.0655 | Val Acc: 0.9900 | Val F1: 0.9900
Epoch 11/15 | Train Loss: 0.0712 | Val Loss: 0.0599 | Val Acc: 0.9867 | Val F1: 0.9866
Epoch 12/15 | Train Loss: 0.0700 | Val Loss: 0.0556 | Val Acc: 0.9883 | Val F1: 0.9883
Epoch 13/15 | Train Loss: 0.0574 | Val Loss: 0.0493 | Val Acc: 0.9900 | Val F1: 0.9900
Epoch 14/15 | Train Loss: 0.0576 | Val Loss: 0.0461 | Val Acc: 0.9933 | Val F1: 0.9933
Epoch 15/15 | Train Loss: 0.0495 | Val Loss: 0.0468 | Val Acc: 0.9917 | Val F1: 0.9917
```

Evaluación en test: accuracy y macro F1

```
In [23]: def evaluate_model(model, loader):
    model.eval()
    y_true = []
    y_pred = []

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images)
            if isinstance(outputs, tuple):
                outputs = outputs[0]

            preds = torch.argmax(outputs, dim=1)
            y_true.extend(labels.numpy())
            y_pred.extend(preds.cpu().numpy())

    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average="macro")

    return acc, f1, y_true, y_pred
```

```
In [24]: resnet_acc, resnet_f1, ytrue_resnet, ypred_resnet = evaluate_model(resnet, test_loader)
inception_acc, inception_f1, ytrue_inception, ypred_inception = evaluate_model(inception, test_loader)
mobilenet_acc, mobilenet_f1, ytrue_mobilenet, ypred_mobilenet = evaluate_model(mobilenet, test_loader)

print("ResNet      -> Accuracy:", resnet_acc, "F1 macro:", resnet_f1)
print("Inception  -> Accuracy:", inception_acc, "F1 macro:", inception_f1)
print("MobileNet  -> Accuracy:", mobilenet_acc, "F1 macro:", mobilenet_f1)
```

```
ResNet      -> Accuracy: 0.995 F1 macro: 0.994976428180195
Inception   -> Accuracy: 0.9816666666666667 F1 macro: 0.9817084368532112
MobileNet   -> Accuracy: 0.9933333333333333 F1 macro: 0.9932965475040602
```

Guardar modelos y medir tamaño en MB

```
In [25]: os.makedirs("modelos_guardados", exist_ok=True)

def save_model_and_get_size(model, filename):
    filepath = os.path.join("modelos_guardados", filename)
    torch.save(model.state_dict(), filepath)
    size_mb = os.path.getsize(filepath) / (1024 * 1024)
    return filepath, size_mb
```

```
In [26]: resnet_path, resnet_size = save_model_and_get_size(resnet, "resnet50_mango.pth")
inception_path, inception_size = save_model_and_get_size(inception, "inceptionv3_mango.pth")
mobilenet_path, mobilenet_size = save_model_and_get_size(mobilenet, "mobilenetv2_mango.pth")

print("ResNet size (MB):", resnet_size)
print("Inception size (MB):", inception_size)
print("MobileNet size (MB):", mobilenet_size)
```

```
ResNet size (MB): 90.04398441314697
Inception size (MB): 93.2837781906128
MobileNet size (MB): 8.75886058807373
```

Tiempo de inferencia

```
In [27]: def inference_time_ms(model, loader, num_images=100):
    model.eval()
    times = []
    processed = 0

    with torch.no_grad():
        for images, _ in loader:
            for i in range(images.size(0)):
                if processed >= num_images:
                    break

                image = images[i].unsqueeze(0).to(device)

                if device.type == "cuda":
                    torch.cuda.synchronize()
                start = time.time()
```

```
        output = model(image)
        if isinstance(output, tuple):
            output = output[0]

        if device.type == "cuda":
            torch.cuda.synchronize()
        end = time.time()

        times.append((end - start) * 1000.0)
        processed += 1

    if processed >= num_images:
        break

    return np.mean(times)
```

```
In [28]: resnet_inf = inference_time_ms(resnet, test_loader, num_images=100)
inception_inf = inference_time_ms(inception, test_loader_inception, num_images=100)
mobilenet_inf = inference_time_ms(mobilenet, test_loader, num_images=100)

print("ResNet inference (ms/img):", resnet_inf)
print("Inception inference (ms/img):", inception_inf)
print("MobileNet inference (ms/img):", mobilenet_inf)
```

```
ResNet inference (ms/img): 9.921104907989502
Inception inference (ms/img): 17.136666774749756
MobileNet inference (ms/img): 8.210549354553223
```

Tabla final de métricas

```
In [29]: results = pd.DataFrame([
    {
        "Modelo": "ResNet50",
        "Accuracy": resnet_acc,
        "F1_macro": resnet_f1,
        "Tamaño_MB": resnet_size,
        "Inferencia_ms": resnet_inf
    },
    {
        "Modelo": "InceptionV3",
        "Accuracy": inception_acc,
        "F1_macro": inception_f1,
        "Tamaño_MB": inception_size,
        "Inferencia_ms": inception_inf
    },
    {
        "Modelo": "MobileNetV2",
        "Accuracy": mobilenet_acc,
        "F1_macro": mobilenet_f1,
        "Tamaño_MB": mobilenet_size,
        "Inferencia_ms": mobilenet_inf
    }
])

results
```

```
Out[29]:
```

	Modelo	Accuracy	F1_macro	Tamaño_MB	Inferencia_ms
0	ResNet50	0.995000	0.994976	90.043984	9.921105
1	InceptionV3	0.981667	0.981708	93.283778	17.136667
2	MobileNetV2	0.993333	0.993297	8.758861	8.210549